UEBA Module for SIEM

Project: UEBA Module for SIEM | Universidade de Aveiro

- Diogo Silva 108212
- Martim Carvalho 108749

- Dataset: X=1 (108212 + 108749 = 216961 → last digit = **1**)

Introduction

UEBA (User and Entity Behavior Analytics)

UEBA is a security approach that builds statistical profiles of normal behaviour from historical data and uses them to flag deviations in new observations. Unlike signature-based detection, which looks for known attack patterns, UEBA detects anomalies that are devices or users behaving in ways inconsistent with their own established baseline. This makes it effective against novel threats and insider attacks that leave no known signature.

GOAL

The goal of this project is to implement a UEBA module. The module reads network flow data, computes per-IP statistical baselines from a training dataset, applies a set of anomaly detection rules to a test dataset, and forwards any flagged alerts to the Wazuh manager via syslog. The full pipeline runs as a single Python script (ueba.py) that produces a consolidated report of anomalous devices and their confidence levels.

Dataset

The assigned dataset is ready to explore. It contains two distinct populations: internal clients (192.168.101.x), representing corporate workstations on the internal network, and external clients (188.83.72.x), representing users accessing a corporate server from outside. Both populations include a training file, used to establish normal behaviour, and a test file, where anomalies are detected.

▷ AI Prompt

> *All the context above was given to the AI exactly as is, just to understand the goal of the project.*

Data Reading & First Observations

Data Loading

The first step was loading the four JSON files and understanding the structure before writing any detection logic. Each file represents one full day of network flows, with 7 fields per record: src_ip, dst_ip, port, timestamp, up_bytes, down_bytes, and protocol.

Code used:

```
int_train = pd.read_json(INTERNAL_TRAIN)
int_test  = pd.read_json(INTERNAL_TEST)
ext_train = pd.read_json(EXTERNAL_TRAIN)
```

```
ext_test  = pd.read_json(EXTERNAL_TEST)
```

Output received:

```
internal_train : (890749, 7)
internal_test  : (1008425, 7)
external_train : (712488, 7)
external_test  : (681696, 7)
```

We concluded that the test files are larger than the training files in both cases.

Protocols and Ports

The first thing we explored after loading was what kind of traffic actually exists in the dataset. Grouping by port across the entire internal training file reveals something immediately striking. Concluded that only two ports exist: TCP/443 (HTTPS) and UDP/53 (DNS). No other protocols, no other ports, anywhere in the dataset. The external dataset contains only TCP/443, no DNS traffic at all, consistent with external clients communicating exclusively with the corporate public server.

Code used:

```
int_train['port'].value_counts()
# 443 (HTTPS)
# 53 (DNS)
```

Every anomaly we detect will be either in HTTPS behaviour or DNS behaviour — there is nothing else to look at. This simplifies the problem significantly and means our detection rules can be precisely targeted at each protocol.

Internal Network Topology

Inspecting the data, grouping the internal data by destination IP, three internal servers emerge consistently across all 198 clients:

Role	IP	Port
DNS Server (primary)	192.168.101.226	53
DNS Server (secondary)	192.168.101.229	53
HTTPS Server	192.168.101.240	443

Every one of the 198 internal clients communicates with all three servers during training with no exceptions. This uniformity is itself a baseline: the network behaves like a corporate environment where all machines follow the same configuration. Any device that deviates from this pattern in the test period stands out immediately.

Internal HTTPS traffic (port 443) does not all go to the same place. Grouping training flows by destination IP reveals two distinct populations: traffic to the internal HTTPS server and traffic to external HTTPS servers.

Code used:

```
https = int_train[int_train['port'] == 443]
# Identify distinct destination IPs in HTTPS traffic
dst_ips = https['dst_ip'].unique()
internal_dsts = [ip for ip in dst_ips if ip.startswith('192.168.')]
print("Internal HTTPS destinations:", internal_dsts)

" Internal HTTPS destinations: ['192.168.101.240'] "
```

The only internal destination is .240, already known from the network topology. Every other destination IP is a public address, confirming a clean two-population split. Per-client stats were then computed separately for each group:

```
INTERNAL = '192.168.101.240'
https_int = https[https['dst_ip'] == INTERNAL]
https_ext = https[https['dst_ip'] != INTERNAL]
for label, grp in [('Internal .240', https_int), ('External', https_ext)]:
  per_ip = grp.groupby('src_ip').agg(up=('up_bytes','sum'), down=('down_bytes','sum'))
  per_ip['ratio'] = per_ip['down'] / per_ip['up']
  print(f"{label}: flows={len(grp):,}  up/client={per_ip['up'].mean()/1e6:.1f} MB
  ↪  ratio={per_ip['ratio'].mean():.4f} ± {per_ip['ratio'].std():.4f}")

" Internal .240:  flows=157,055  up/client=9.0 MB   ratio=9.2255 ± 0.5809 "
" External HTTPS: flows=627,522  up/client=36.1 MB  ratio=9.2460 ± 0.2511 "
```

Both groups were characterised separately:

Destination	Flows	Up (mean/client)	Down (mean/client)	Ratio mean	Ratio std
Internal server .240	157,055	9.0 MB	83.2 MB	9.2255	0.5809
External HTTPS servers	627,522	36.1 MB	333.6 MB	9.2460	0.2511

External HTTPS servers account for 80% of all HTTPS flows. Both groups maintain nearly identical down/up ratios (∼9.23 vs ∼9.25) that means the clients download roughly 9.2 bytes for every byte they upload, a heavily download-dominant pattern typical of web browsing.

The std is computed across all 198 clients, it measures how much each individual client's ratio deviates from the group mean. The external HTTPS traffic is tighter (0.25) than traffic to .240 (0.58), confirming the two groups are statistically homogeneous on this dimension.

This confirms that the ratio is a structural property of the connection type rather than a destination-specific artefact. The combined per-client upload mean of ∼45 MB is the direct sum of the two groups (9 MB to .240 + 36 MB to external). This validates using the combined aggregate as the baseline for the HTTPS exfiltration rule, the two populations are statistically homogeneous on the ratio dimension.

DNS Invariant

The most important single observation in the entire dataset came from inspecting DNS destination IPs. During training, every DNS query from every internal client goes exclusively to .226 or .229, without a single exception across all 198 clients and 890K rows.

Code used:

```
dns = int_train[int_train['port'] == 53]
internal_servers = set(dns['dst_ip'].unique())
# {'192.168.101.226', '192.168.101.229'}
```

Output received:

```
Internal DNS servers: {'192.168.101.229', '192.168.101.226'}
```

This is an invariant, not a tendency. It directly motivates one of the DNS detection sub-rules: any internal client that sends a DNS query to an IP outside this set during the test period is immediately and unconditionally anomalous. No threshold needed, no baseline this is just a violation of a rule that held perfectly across the entire training dataset.

External Clients

The external dataset contains clients from the 188.83.72.x subnet connecting to a corporate public server over HTTPS only, no DNS traffic exists in the external files. During training, these clients show a remarkably consistent down/up byte ratio across all 196 devices:

Metric	Value
Mean down/up ratio	8.50
Std	0.04
Min	8.40
Max	8.61

A standard deviation of 0.04 on a ratio of 8.50 means each client downloads almost exactly 8.5× what it uploads, consistently, across every external user. This is the tightest baseline in the entire dataset. It tells us the corporate server behaves very predictably which means any external client that breaks this pattern in the test period is doing something genuinely unusual, not just natural variation. A 3σ window of [8.38, 8.62] is enough to catch real anomalies with very few false positives.

The script also characterised the external inter-flow intervals during baseline computation:

Metric	Value
Mean	8.076 s
Median	1.040 s
Std	118.794 s
p90	1.910 s
p95	34.750 s

The gap between median (1.04 s) and mean (8.08 s), and the jump from p90 (1.91 s) to p95 (34.75 s), confirms a heavy-tailed distribution, consistent with human web browsing: short active bursts separated by reading pauses, with occasional long overnight idle gaps.

▷ AI Prompt

Explore the data given in the dataset1 folder having in mind all the context I gave you before. The script should read the training data and output useful information that we could use in order to achieve our goal. The idea is to use a Python script, using the provided example (sampleScript.py) as a base.

Baseline Computation

Before any detection logic runs, compute_baselines() reads both training files and builds a dictionary of statistical profiles that every rule will consume. This function computes per-IP aggregates (total upload, DNS flow counts, inter-flow intervals, destination countries) and derives the thresholds that define "normal" for each detection rule. Nothing is flagged here. The goal is purely to characterise the training period so the test period can be compared against it. The function follows a single design principle: compute everything once, return it all as a dictionary. The naive alternative would be to re-read and re-aggregate the training data inside each rule function.

This means scanning ∼900k rows five separate times and duplicating the same groupby operations. Instead, compute_baselines() runs exactly once at startup, builds every statistic every rule will ever need, and passes the result as a single dict b. Each rule receives b as its only input and reads from it without modifying it.

The function itself has no side effects, it prints nothing, flags nothing, and touches no external state. This makes it independently testable and means that replacing the baseline source requires changing only this one function, with zero impact on any detection logic.

Code Implementation

```
# HTTPS: group port-443 flows per client, compute total upload per IP
  https = int_train[int_train['port'] == 443]
  https_per_ip = https.groupby('src_ip').agg(total_up=('up_bytes','sum'),
↪  total_down=('down_bytes','sum'))

# BotNet: compute per-protocol inter-flow interval std per client
  https_sorted = https_train.sort_values(['src_ip','timestamp'])
  https_sorted['interval'] = https_sorted.groupby('src_ip')['timestamp'].diff()
  https_interval_std_per_ip = https_sorted.groupby('src_ip')['interval'].std()

  dns_sorted = dns_train.sort_values(['src_ip','timestamp'])
  dns_sorted['interval'] = dns_sorted.groupby('src_ip')['timestamp'].diff()

# Geo/Destination: build global sets - union of all ASNs and internal dst_ips seen by any
↪  client in training
  global_train_asns = set(pub_train['dst_ip'].apply(get_asn).dropna())
  global_train_internal_dsts = set(priv_train['dst_ip'].unique())

# Fan-out: unique external dst_ip count per src_ip (Step 4c)
  ext_fan = pub_train.groupby('src_ip')['dst_ip'].nunique()

# New source IPs: set of all src_ips seen in training (Step 1)
  train_src_ips = set(int_train['src_ip'].unique())
```

```
# DNS: count flows per client, extract the set of destination servers
  dns_per_ip = dns.groupby('src_ip').agg(dns_flows=('up_bytes','count'))
  dns_internal_servers = set(dns['dst_ip'].unique())
```

The dictionary compute_baselines() returns one key per metric below. Each rule function receives this dictionary as its only input and reads the relevant keys no rule recomputes or re-reads training data on its own.

Threshold table

All thresholds are derived dynamically from the training data. The script prints them at startup:

```
New source IP:                 none - binary set membership check
HTTPS upload threshold:        116.6 MB         (mean+3σ)
DNS flow threshold:            1399 flows       (mean+3σ)
BotNet HTTPS interval p05:     18.81 s
BotNet DNS  interval p05:      63.73 s
External ratio window:         [8.3817, 8.6226]
Geo min intensity flows:       10 flows
External fan-out threshold:    258 unique dst IPs  (mean+3σ)
```

Each threshold targets a different dimension of behaviour. Not every rule requires a statistical threshold: when a training invariant holds absolutely, any test-period violation is unconditionally anomalous.

- **New source IP (Step 1)** requires no threshold at all. Any internal device that generates traffic in the test period but had zero presence during training is flagged unconditionally. The only computation is a set membership check against all source IPs observed in training.
- **HTTPS upload threshold (116.6 MB)** defines the maximum total data a normal internal client sends over HTTPS in a day; anything above this suggests bulk data exfiltration. The threshold is derived from combined HTTPS traffic (internal .240 + external servers). Both groups have identical ratios ($\sim$9.23), so the combined aggregate is a valid baseline.
- **DNS flow count threshold (1,399 flows)** flags clients generating an abnormal number of DNS queries, the primary signal for DNS-based C&C beaconing.
- **BotNet interval p05 thresholds** are split by protocol: 18.81 s for HTTPS and 63.73 s for DNS, representing the 5th percentiles of per-client interval standard deviations. Any client below its protocol's floor is more regular than 95% of all legitimate training clients. Separating the protocols prevents a device with naturally irregular HTTPS activity from masking a tight DNS beaconing pattern.
- **External ratio window ([8.3817, 8.6226])** defines the expected down/up range for external clients. Any external client outside this window is interacting with the corporate server in an anomalous way.
- **Geo minimum intensity floor (10 flows)** filters CDN noise: any client reaching a country new to the entire network with fewer than 10 flows is treated as a CDN edge rotation rather than a deliberate new destination.
- **External fan-out threshold (258 unique IPs)** flags any internal client contacting an anomalous number of distinct external destinations in a single day. Derived from mean (118.1)

+ 3σ (3×46.7), it catches sweep-style behaviour where a host checks in with many scattered external endpoints.

The 3σ Rule

The choice of 3σ as the threshold is an industry standard in statistical anomaly detection. Under a normal distribution, 99.9% of observations fall within 3 standard deviations of the mean, meaning only 0.1% of legitimate traffic would be flagged as anomalous by chance.

Applied consistently across all volume-based rules, it means the thresholds adapt automatically to the dataset: a network with higher baseline DNS traffic will produce a higher DNS threshold, and a network with tighter HTTPS upload patterns will produce a lower upload threshold. No manual tuning required. To make this concrete for this dataset: with 198 internal clients, a 0.1% false-positive rate per rule means roughly 0.2 false positives per rule per day in the worst case, easily manageable for a security analyst.

The alternatives were evaluated and rejected:

- **2σ** (97.7% coverage) would produce approximately 4 false positives per rule per day, creating a noise floor that drowns real signals.
- **4σ** (99.9937% coverage) would set the bar so high that moderate exfiltration (a device uploading 200 MB when the mean is 45 MB) would go undetected.

Country Distribution

As part of the baseline computation, every internal client's HTTPS traffic was mapped to destination countries using a GeoIP database. Across all 198 clients and the full training period, the network contacted 36 unique countries. The top 5 alone account for 94.8% of all flows, showing a heavily concentrated geographic footprint:

Country	Flows	% of Total
PT	234442	37.4%
US	164761	26.3%
CA	93719	14.9%
FR	64307	10.2%
NL	37617	6.0%
(others)	32676	5.2%

These 36 countries define the known geographic footprint of this network, any country contacted in the test period that never appeared in training is an immediate network-level anomaly, regardless of which client triggered it.

▷ AI Prompt

Now we need to implement a `compute_baselines()` function to define the baselines. The idea is to compute the necessary statistics and thresholds that will be used by the detection rules. Use the information gathered during data exploration and the list of rules to decide which statistics are relevant for each rule. The goal is to find the "normal" thresholds from the training data, so that when we apply the rules to the test data, any deviations from these baselines can be flagged as anomalies.

Rule Implementation

With the baselines established, the next step was implementing the detection rules. Each rule targets a specific type of anomalous behaviour and operates independently, consuming the baseline dictionary computed in the previous step. Each rule follows the same pipeline: statistical thresholds are derived exclusively from the training dataset and then applied to the test dataset, which represents a separate day where anomalous behaviour may be present. The training data never sees the test data, and the test data never influences the thresholds.

This strict separation is what gives the detection statistical validity: the baselines describe what normal looks like before any anomaly occurs, and the rules measure how far the test behaviour deviates from that established normal. The rules were designed to be complementary rather than redundant: some catch high-volume bulk attacks, others catch low-and-slow patterns that volume alone would miss entirely. For each rule, the process followed the same structure: define the metric, compute the threshold from training, apply it to the test data, and examine what gets flagged.

In several cases the first approach produced too many false positives or broke down statistically, requiring iteration before arriving at a clean result.

Rule Design Rationale

The rules are designed around the project's five detection requirements. The table below maps each requirement to the implemented step(s) and the signal dimension each measures.

Spec Requirement	Step(s)	Signal dimension
(i) BotNet activities	Step 6	Inter-flow timing regularity per protocol
(ii) HTTPS data exfiltration	Step 3	Upload volume
(ii) DNS data exfiltration	Step 5 sub-rule 1	DNS query volume
(iii) C&C activities using DNS	Step 5 sub-rule 2 + Step 6 (DNS)	Resolver bypass + timing regularity
(iv) Anomalous external destinations	Steps 4, 4b, 4c	Geography, ASN infrastructure, unique IP count
(v) External users anomalous	Step 2	Down/up byte ratio
(bonus) Rogue/new devices	Step 1	Set membership — no threshold needed

Steps 4, 4b, and 4c address the same threat category from three independent dimensions: new countries (geography), new ASNs (infrastructure provider), and anomalous unique destination count (breadth). Splitting them ensures that a compromise staying within known geography but using new hosting, or staying within known infrastructure but sweeping unusually many endpoints, is still caught.

Step 5 and Step 6 both target DNS-based threats but measure different things: Step 5 measures query volume (how many), Step 6 measures timing regularity (how evenly spaced). A bursty DNS tunnel can have high volume without tight timing; a slow-rate beacon can have tight timing without high volume. Step 1 is a bonus rule beyond the spec minimum, added after discovering that two devices in the test set generate thousands of flows while staying deliberately sub-threshold on every statistical metric — the only detection that cannot be evaded by volume calibration is a binary set membership check.

New Source IP Detection

What to look for

The most fundamental principle in UEBA is that a behavioral baseline can only describe devices that were actually observed during training. Every statistical rule in this pipeline — upload volume, DNS count, beaconing intervals, fan-out — produces a number that is compared against a distribution computed from training data. But this comparison is only meaningful if the device itself has a training history. A device with zero training presence has no baseline to deviate from, not a small deviation, but a complete absence of normal.

In a corporate environment, every legitimate device should appear in the training window. A new device appearing in the test period is inherently suspicious: it could be a rogue endpoint physically connected to the network, a new implant deployed by an attacker after the training window, or a compromised host that changed its IP to evade rules tuned on the original identity. Any of these scenarios warrants immediate investigation regardless of what the device is actually doing in terms of volume or timing.

This rule requires no statistics and no threshold. The question is binary: was this source IP seen at all during training?

Code Implementation

```
train_ips = baselines['train_src_ips']                    # set of all training src_ips
new_ips   = set(int_test['src_ip'].unique()) - train_ips
```

One set subtraction. No mean, no standard deviation, no threshold tuning. A device is either in the training set or it is not. This simplicity is also what makes it immune to the false positive problems that affect threshold-based rules: there is no σ to miscalibrate and no distribution to mis-model.

Results

The rule flagged 2 internal IPs:

```
[ALERT] 192.168.101.11
  Why: 1508 flows (1309 HTTPS, 199 DNS), HTTPS upload: 14.3 MB,
       unique ext IPs: 64 - all metrics sub-threshold,
       absent entire training period
[ALERT] 192.168.101.93
  Why: 3788 flows (3286 HTTPS, 502 DNS), HTTPS upload: 38.0 MB,
       unique ext IPs: 101 - all metrics sub-threshold,
       absent entire training period
```

Both devices generate thousands of flows in the test period yet were completely invisible to every other rule in the pipeline. The reason is instructive: their traffic profiles are deliberately sub-threshold. .11 uploads only 14.3 MB over HTTPS (threshold: 116.6 MB), contacts only 64 unique external IPs (threshold: 258), and generates only 199 DNS queries (threshold: 1,399). .93 uploads 38 MB, contacts 101 unique external IPs, and generates 502 DNS queries. Every volume metric sits well below every threshold. This is the profile of a stealthy implant: generate enough traffic to operate, but stay below every detection limit on every dimension. The only rule that catches them is the one that requires no threshold at all.

A complementary observation: the training set contains 2 IPs that disappeared entirely in the test period — 192.168.101.57 (2,117 training flows) and 192.168.101.202 (2,501 training flows). While device turnover is normal and these are not flagged as alerts, the simultaneous disappearance of two training devices and appearance of two new ones in the same test window is a pattern consistent with IP reassignment following lateral movement. It is noted here as investigative context for an analyst reviewing the full picture.

External Anomaly Detection

What to look for

The external anomaly rule targets clients from the 188.83.72.x subnet accessing the corporate server in an unusual way. The key insight here is that the relevant signal is not the amount of traffic, but the ratio between downloaded and uploaded bytes. During training, every external client consistently downloads roughly 8.5× what it uploads: a tight, stable pattern with a standard deviation of just 0.04.

This makes the ratio the most discriminating metric in the entire dataset: a client whose ratio breaks this pattern is interacting with the server in a fundamentally different way than normal, regardless of total volume. A 3σ window of [8.3817, 8.6226] is sufficient to flag real anomalies with minimal false positives.

Code Implementation

Code used:

```
low = mean - SIGMA * std  # 8.3817
high = mean + SIGMA * std # 8.6226

flagged = per_ip[(per_ip['ratio'] < low) | (per_ip['ratio'] > high)]
```

We compute the ratio for each external client in the test period and flag anyone outside the 3σ window. Below the lower bound means the client is uploading proportionally more than expected. Above the upper bound means it is downloading proportionally more than expected.

Results

The rule flagged 5 external IPs, which naturally split into two distinct behavioural groups:

```
[ALERT] 188.83.72.61   ratio=8.232  deviation=6.7σ
[ALERT] 188.83.72.64   ratio=8.338  deviation=4.1σ
[ALERT] 188.83.72.174  ratio=8.334  deviation=4.2σ
[ALERT] 188.83.72.182  ratio=8.661  deviation=4.0σ
[ALERT] 188.83.72.210  ratio=8.644  deviation=3.5σ
```

The results revealed two groups:

- The first group (.61, .64, .174) sit below the lower bound. Their ratio is lower than normal, meaning they upload proportionally more than a legitimate client would. This is consistent with a compromised account being used to push data toward the corporate server, or with reversed exfiltration where data is staged on the server from outside.

- The second group (.182, .210) sit above the upper bound, downloading more than expected relative to what they upload, consistent with bulk data retrieval or unusual large-object access. These are two different threat models, but the same symmetric rule catches both because both break the tight ratio invariant in opposite directions.

The 188.83.72.61 at 6.7σ is the most extreme anomaly in the entire external dataset.

Internal Anomaly Detection (HTTPS Exfiltration)

What to look for

The HTTPS exfiltration rule targets internal clients sending an abnormal amount of data outbound over HTTPS. The detection metric is total upload volume per client: any client uploading more than mean + 3σ (116.6 MB) in the test period is flagged. The combined HTTPS traffic (internal .240 + external servers) is used as the baseline, validated by the near-identical per-group ratios ($\sim$9.23 in both) characterised during exploration.

A second signal, the PCR (Producer-Consumer Ratio), was implemented and evaluated before finalising this rule. PCR = (up – down) / (up + down) measures the upload/download balance independently of volume. The training mean was –0.8046 (heavily download-dominant), and the 3σ threshold was –0.7918. Applied to the test data, PCR flagged four additional IPs with upload volumes well below the training mean. On closer inspection, these were false positives: with few HTTPS flows, natural variance in the upload/download balance is enough to push PCR across the threshold. PCR was removed; the devices it would have caught are independently and more reliably detected by the BotNet beaconing rule through timing analysis.

Code Implementation

```
flagged = per_ip['total_up'] > vol_threshold  # > 116.6 MB
```

The threshold is derived from combined HTTPS traffic (internal .240 + external servers). Both groups show near-identical down/up ratios ($\sim$9.23), confirming the combined aggregate is a valid baseline for volume normalisation.

One metric was evaluated and removed before arriving at this design: the raw down/up ratio (total_down / total_up), which is the same metric used for external clients. An equivalent internal ratio was computed and applied to the test data. It flagged the identical set of IPs as the volume threshold: zero additional detections. The raw ratio approach was dropped in favour of the simpler volume threshold alone.

Results

The rule flagged 6 internal IPs:

```
[ALERT] 192.168.101.187  upload=7586 MB  deviation=316σ
[ALERT] 192.168.101.14   upload=5352 MB  deviation=223σ
[ALERT] 192.168.101.208  upload=4402 MB  deviation=183σ
[ALERT] 192.168.101.26   upload=259 MB   deviation=9σ
[ALERT] 192.168.101.197  upload=138 MB   deviation=4σ
[ALERT] 192.168.101.207  upload=119 MB   deviation=3σ
```

All six devices exceed the 116.6 MB threshold cleanly. The top three (.187, .14, .208) represent bulk data dumps at 4.4–7.6 GB, 183–316σ above baseline — orders of magnitude beyond any legitimate client in training. The remaining three (.26, .197, .207) sit between 119 MB and 259 MB, each clearly distinguishable from the highest unflagged device at 105.2 MB.

GeoDestination Anomaly Detection

What to look for

The geo destination rule targets internal clients contacting countries that are new to the entire network. The first approach was straightforward: flag any internal client that contacts a country in the test period that it never contacted during training.

The result was immediate and catastrophic: 163 of 198 clients flagged, an 82% false positive rate. The reason is CDN rotation: large content delivery networks distribute their infrastructure globally and rotate IP addresses across countries regularly, meaning a legitimate user visiting the same website on two different days may hit servers in different countries each time. A naive per-IP new-country rule is completely blind to this and produces noise, not signal.

Code Implementation

To solve the false positive problem, the naive per-IP approach was replaced with a global detection strategy: flag any client reaching a country that no client in the entire network contacted during training. Any access to such a country is immediately anomalous at the network level — not just new to the individual device, but new to the whole organisation.

```
new_to_network = test_countries - global_train_countries

if new_to_network and flows_to_new >= MIN_INTENSITY_FLOWS:
      # fire alert
```

The set difference operation is the core: subtracting the network-wide known country set from the observed destinations leaves only genuinely new territory. A MIN_INTENSITY_FLOWS = 10 filter removes CDN edge noise — any new-country access with fewer than 10 flows is treated as a stray CDN rotation rather than a deliberate connection.

ASN (Autonomous System Number) detection was also implemented as a complementary rule: a device contacting a new cloud provider or hosting company is suspicious even if the destination country is already in the training set. ASN lookups use a separate GeoIP database and compare against the global union of all ASNs seen by any client during training. This is implemented as a separate step (Step 4b) rather than merged into this rule, since it catches a different threat model — new infrastructure, not just new geography.

Results

The global new-country rule with the 10-flow intensity filter produced 3 flagged IPs:

```
[ALERT] 192.168.101.125   523 flows to new country/ies:
          BG, CZ, DK, FI, IR, LV, PL, PY, RU, SC, UA
[ALERT] 192.168.101.36    904 flows to new country/ies:
          AR, BA, BG, BY, CZ, EE, FI, GE, HU, IQ, IR, KZ, LU, LV, PL, RU, UA
[ALERT] 192.168.101.72   1157 flows to new country/ies:
```

BD, BY, BZ, EE, FI, GR, HR, IR, KZ, LU, LV, NG, PL, RU, UA

All three are high-confidence detections: they each reach multiple countries new to the entire network, including Russia, Iran, and Ukraine — geographies absent from the 36 countries seen across all 198 training clients. Each has hundreds of flows to the new destinations, well above the 10-flow noise floor. All three are also independently confirmed by the New Destination rule (Step 4b), which detects the same infrastructure through ASN analysis.

New Destination Detection

What to look for

Beyond new geographic territory, two complementary destination-based signals catch compromised devices reaching infrastructure that was simply never contacted during training — regardless of country.

- Sub-rule A (New External ASN): flags any internal client contacting an Autonomous System (cloud provider, hosting company, ISP) that no client in the entire network used during training. Country detection can miss this if the new infrastructure happens to be in a country the network already visited. Comparing against the global union of all ASNs seen across all 198 training clients makes the signal noise-resistant: legitimate CDN providers used by even a single training client are whitelisted, so only genuinely unseen infrastructure triggers an alert.
- Sub-rule B (New Internal Destination): flags any internal client communicating with an internal IP (192.168.101.x) that was never a destination in any training flow. During training, all internal HTTPS traffic goes exclusively to the single known server .240. Any internal-to-internal TCP:443 traffic to any other address is therefore an absolute violation — the invariant held perfectly across all 198 clients and 890 K training rows.

Code Implementation

```
# Sub-rule A - new external ASN (global training set)
pub_test['asn'] = pub_test['dst_ip'].apply(get_asn)
new_asns = set(grp['asn'].dropna().unique()) - global_train_asns
if new_asns and flows >= MIN_NEW_EXTERNAL_FLOWS:  # MIN = 10
    # fire alert

# Sub-rule B - new internal destination (global training set)
new_dsts = set(grp['dst_ip'].unique()) - global_train_internal_dsts
if new_dsts and flows >= MIN_NEW_INTERNAL_FLOWS:  # MIN = 5
    # fire alert
```

The global baseline comparison is the critical design choice. Using per-client known-ASN sets instead would cause false positive explosions: CDN providers rotate IP addresses across hundreds of ASNs, so nearly every device appears to reach "new" ASNs compared to its own individual training history. The global union approach is noise-immune because any ASN touched by any of the 198 training clients is whitelisted.

Results

Step 4b flagged 6 alerts across 6 IPs:

```
[ALERT] 192.168.101.36    1015 flows to 287 new-to-network ASNs
```

```
          (incl. Russian hosting: REG.RU, C.S.T Ltd, Eneva Ltd...)
[ALERT] 192.168.101.72    1319 flows to 354 new-to-network ASNs
          (incl. Cloud Technologies LLC/Cloud.ru, AKT LLC, NPO Unimach LLC...)
[ALERT] 192.168.101.125    598 flows to 169 new-to-network ASNs
          (incl. OBIT Ltd, AS43668 LLC, REG.RU...)
[ALERT] 192.168.101.68     273 flows to new internal IPs:
          192.168.101.138, 192.168.101.186
[ALERT] 192.168.101.138    209 flows to new internal IPs:
          192.168.101.186, 192.168.101.68
[ALERT] 192.168.101.186    429 flows to new internal IPs:
          192.168.101.138, 192.168.101.68
```

The ASN detections for .36, .72, and .125 independently confirm the geo anomalies already identified in Step 4: the same three devices reach both new countries and new infrastructure, with the ASN names pointing directly at Russian hosting providers not present anywhere in training. This cross-rule corroboration promotes all three to HIGH confidence.

The lateral movement triangle (.68, .138, .186) is the most significant finding in this rule. These three internal devices communicate with each other over TCP:443 — a connection type that, across all 198 clients and 890 K training rows, goes exclusively to the known server .240. The fact that all three devices appear both as sources and destinations, forming a closed triangle of mutual communication, rules out a simple misconfiguration. The traffic pattern is symmetric (up/down byte ratios near 1:1, unlike the 9:1 ratio of normal web browsing), consistent with peer-to-peer command relay or lateral file transfer between compromised hosts.

External Destination Fan-out

What to look for

The fan-out rule addresses a different dimension of suspicious external behaviour: not what destinations a client reaches, but how many. During training, internal clients contact a mean of 118.1 unique external IPs per day, with a standard deviation of 46.7. This gives a tight upper boundary — legitimate browsing concentrates traffic on a small set of content servers, CDNs, and cloud providers. A device that suddenly reaches hundreds more unique external endpoints than any legitimate client did in training is not browsing normally; it is sweeping through scattered infrastructure, consistent with a C2 implant performing check-ins across a distributed botnet panel or a scanning probe probing a wide address range.

The key distinction from Step 4 and Step 4b is the signal being measured. Step 4 flags new geography; Step 4b flags new ASNs (new infrastructure providers). Step 4c is purely statistical: it fires when the raw count of unique external destinations is anomalous, regardless of whether those destinations are known or new. A device could contact only known ASNs in known countries and still fire this rule if it contacts far more of them than any training client ever did.

Code Implementation

```
ext_fan =
↪  int_test[~int_test['dst_ip'].apply(is_private)].groupby('src_ip')['dst_ip'].nunique()
threshold = ext_fan_mean + SIGMA * ext_fan_std  # 258 unique IPs

flagged = ext_fan[ext_fan > threshold]
```

The implementation is deliberately simple: one groupby, one nunique, one threshold comparison. The baseline is the per-client unique external IP count from training (mean=118.1, std=46.7), derived from the same public-IP subset already used by the geo and ASN rules.

Results

The rule flagged 4 internal IPs:

```
[ALERT] 192.168.101.72    811 unique external IPs
          (14.9σ above mean 118; threshold 258)
[ALERT] 192.168.101.36    639 unique external IPs
          (11.2σ above mean 118; threshold 258)
[ALERT] 192.168.101.125   409 unique external IPs
          (6.2σ above mean 118; threshold 258)
[ALERT] 192.168.101.207   273 unique external IPs
          (3.3σ above mean 118; threshold 258)
```

All four are already flagged by other rules, and that is precisely the point: fan-out provides a third independent confirmation for .36, .72, and .125, and a second independent axis of evidence for .207 alongside its exfiltration and DNS signals. The deviations are substantial — .72 contacts nearly 7× more unique external IPs than the training mean — reinforcing that these devices are not exhibiting borderline behaviour but a fundamentally different operational mode. The highest unflagged device is .114 at 234 unique IPs, sitting 24 IPs below the threshold with no other anomalous signals, confirming the threshold is not set too aggressively.

DNS Anomaly Detection

What to look for

The DNS anomaly rule targets internal clients exhibiting abnormal DNS behaviour, using two independent sub-rules that catch different aspects of the same threat category.

- DNS-1 (Volume) flags any client whose total DNS flow count exceeds mean + 3σ (1,399 flows): an extreme query rate is the primary signal for both DNS-based C&C beaconing and DNS tunneling.
- DNS-2 (Public server) flags any client that sends a DNS query to a server outside .226 and .229. Since this invariant held perfectly across all 198 clients during training, any violation is unconditionally anomalous and requires no statistical threshold.

The two sub-rules are complementary: DNS-1 catches volume-based abuse through the internal resolvers, DNS-2 catches any attempt to bypass them entirely:

```
# DNS-1: volume anomaly
  threshold      = mean + SIGMA * std          # 1,399 flows
  volume_flagged = dns_count[dns_count > threshold]
# DNS-2: public DNS server (binary invariant)
  public_dns = dns_test[~dns_test['dst_ip'].isin(internal_servers)]
```

C&C Beaconing vs DNS Exfiltration

Before presenting the results, it is important to distinguish between two different DNS-based attack patterns, since they look different in the data and have different implications.

- DNS data exfiltration encodes stolen data inside DNS query subdomains: the queries are bursty, aperiodic, and contain long high-entropy subdomain labels.
- DNS C&C beaconing uses DNS queries as a heartbeat mechanism: a malware implant checks in with its C&C server at a fixed clock-driven interval, producing a very regular, high-volume query pattern with normal-looking short domain names.

Aspect	C&C Beaconing	DNS Exfiltration
Pattern	Fixed periodic intervals	Bursty, aperiodic
Query content	Short, normal-looking	Long high-entropy subdomains
Volume	Extreme query count	Moderate count, large payload
Interval	Exact, clock-driven	Irregular

What we detected in this dataset is C&C beaconing, not exfiltration: the exact 0.05s median interval across all flagged devices is the signature of a malware timer. Actual data exfiltration in this dataset happens over HTTPS, as seen in the previous rule.

We initially considered flagging DNS exfiltration by FQDN entropy. However, the dataset does not include subdomain fields, only destination IPs, making entropy analysis impossible with the available data.

A second approach was also attempted before arriving at the final: flagging anomalous DNS behaviour by per-query payload size (up_bytes). The hypothesis was that DNS exfiltration encodes data inside subdomain labels, making each query packet larger than a normal lookup. The training baseline for DNS query size is tight, applied to the test data, not a single IP exceeded this threshold. The beaconing pattern in this dataset uses fixed-size queries carrying only a short beacon identifier, not encoded data, so the payload size is indistinguishable from normal traffic. This sub-rule was computed, tested, and dropped, it has zero discrimination power for this dataset.

Results

DNS-1 flagged 5 internal IPs. DNS-2 produced zero alerts — no client in the test period sent DNS queries to a public server:

```
[ALERT] 192.168.101.41   flows=39,493  median interval=0.05s  increase=129×
[ALERT] 192.168.101.23   flows=8,651   median interval=0.05s  increase=8.6×
[ALERT] 192.168.101.201  flows=2,941   median interval=0.05s  increase=15.7×
[ALERT] 192.168.101.148  flows=1,661   median interval=0.05s  increase=4.8×
[ALERT] 192.168.101.207  flows=1,418   median interval=0.05s  increase=3.4×
```

Every flagged device shows an exact 0.05s median interval between DNS queries: a clock-driven malware timer firing consistently throughout the day. The .41 is the most extreme case at 39,493 flows, 129× its own training baseline, with queries concentrated almost entirely on the two internal resolvers.

The zero result for DNS-2 is not a failure of the rule, it is explained by the beaconing pattern itself. The malware routes its C&C traffic through the internal DNS resolvers, which forward the queries upstream to the external C2 server. The infected devices never contact public DNS directly, which is why DNS-2 sees nothing. Both results together paint a consistent picture.

BotNet Beacon Detection

What to look for

The BotNet beaconing rule targets internal clients whose network traffic follows an unusually regular timing pattern. A botnet implant checks in with its C&C server at a fixed interval, producing a very low standard deviation in inter-flow timestamps compared to legitimate users, whose traffic is naturally irregular.

The detection metric is the standard deviation of inter-flow intervals per client, computed separately for HTTPS traffic and DNS traffic. Separating the protocols is essential: a device with irregular HTTPS browsing and regular DNS beaconing would have its DNS signal masked by the HTTPS noise in a combined calculation.

Mean – 3σ cannot be applied: both per-protocol interval std distributions are right-skewed (a long tail of high-variance clients pulls the mean up), so the formula produces a negative lower bound for HTTPS and is equally inapplicable for DNS. No meaningful floor can be set this way on a right-skewed distribution.

Code Implementation

The threshold for each protocol is its 5th percentile (p05) of per-client interval standard deviations from training. This is the empirical lower boundary of normal behaviour: any client in the test period with an interval std below the HTTPS floor (18.81 s) is more regular than 95% of all legitimate HTTPS sessions in training; similarly for DNS (63.73 s).

For HTTPS beaconing, a regularisation ratio check is added on top of the threshold: the device's own training-period HTTPS interval std must be at least 1.5× higher than its test-period std. This guards against devices that were naturally tight in training from being falsely flagged. A device with train_std = 20.00 s and test_std = 18.00 s has barely changed; a device with train_std = 120.00 s and test_std = 18.00 s has genuinely become more regular — only the latter is beaconing.

DNS beaconing includes an additional volume guard: the DNS flow count must also exceed the DNS volume threshold (1,399 flows). A device with few but regularly-timed DNS queries is not generating enough traffic to constitute meaningful beaconing.

An alternative approach, the Coefficient of Variation (CV = std/mean, with CV ≤ 0.2), was evaluated and rejected: the CV ranges for normal clients (3.93–25.52) and confirmed beacons (3.7–16.6) overlap almost completely in training, providing no clean separation point.

```
# HTTPS beaconing: below p05 AND regularisation ratio ≥ 1.5×
https_std < https_p05  and  (train_std / https_std) >= 1.5

# DNS beaconing: below p05 AND volume anomaly
dns_std < dns_p05  and  dns_count > dns_vol_threshold
```

Results

The rule flagged 6 internal IPs, split into two per-protocol beaconing channels:

```
[ALERT] 192.168.101.23   DNS  interval std=9.13s   (threshold 63.73s)  median=0.05s
[ALERT] 192.168.101.41   DNS  interval std=12.60s  (threshold 63.73s)  median=0.05s
[ALERT] 192.168.101.201  DNS  interval std=36.62s  (threshold 63.73s)  median=0.05s
```

```
[ALERT] 192.168.101.117  HTTPS interval std=10.35s (threshold 18.81s)
          median=1.00s  regularisation=8.46×
[ALERT] 192.168.101.157  HTTPS interval std=18.64s (threshold 18.81s)
          median=1.03s  regularisation=6.49×
[ALERT] 192.168.101.188  HTTPS interval std=18.27s (threshold 18.81s)
          median=1.03s  regularisation=1.84×
```

The 6 devices split cleanly by protocol:

- The DNS beacons (.23, .41, .201) fire at an exact 0.05 s median interval through the internal DNS resolvers, consistent with Step 5's findings. The same malware heartbeat confirmed by both volume and timing analysis.
- The HTTPS beacons (.117, .157, .188) fire at intervals clustering tightly between 1.00 and 1.03 seconds — the fingerprint of a single malware family sharing the same hardcoded C2 timer.

The regularisation ratio validates that each HTTPS beacon genuinely changed behaviour in the test period. .117 shows the most dramatic change: its training HTTPS interval std was ∼87.40 s (irregular browsing); in the test period it dropped to 10.35 s (8.46× more regular). .157 shows a 6.49× change. Even the borderline .188 shows a 1.84× change, comfortably above the 1.5× threshold.

Two previously considered devices were correctly excluded by the per-protocol approach. .32 and .160 appeared suspicious under a combined-channel analysis because mixing their DNS 0.05 s intervals with HTTPS ∼1.00 s intervals produced an artificially low overall std. With per-protocol analysis, .32's DNS flow count falls below the 1,399-flow volume guard, and .160's HTTPS regularisation ratio is only 1.17× — barely changed from training, not a genuine behavioural shift.

The 192.168.101.148, flagged by DNS-1 with 1,661 flows at a 0.05 s median interval, is not flagged by this rule. Although its DNS query count exceeds the volume threshold, its DNS interval std is above the 63.73 s p05 floor: the queries arrive in bursts rather than uniformly spaced, so the timing alone does not meet the beaconing definition. Step 5 and Step 6 together cover both cases — volume-anomalous DNS and timing-regular beacons are complementary, not redundant signals.

Final Results

Final output

Running the full detection pipeline against the test dataset produced 26 unique anomalous IPs: 21 internal clients (192.168.101.x) and 5 external clients (188.83.72.x). Each flagged IP was assigned a confidence level based on how many independent rules triggered it: HIGH when two or more rules agree on the same device, MEDIUM when only one rule fires.

The two-level scheme is not arbitrary:

- When a single rule flags a device, there is always a small residual probability of a false positive.
- When two completely independent detection methods both flag the same IP on the same day, the probability of a coincidental double false positive is the product of the individual rates, dropping to approximately 0.0001% per device.

The 7 HIGH confidence IPs are therefore treated as confirmed compromised devices. The 19 MEDIUM confidence IPs are genuine anomalies that warrant investigation but cannot be confirmed by corroboration alone.

```
Total Unique Anomalous IPs: 26
   - Internal (192.168.101.x): 21
   - External (188.83.72.x): 5

 -- HIGH confidence (2+ independent rules) --
 [HIGH  ] 192.168.101.23        DNS Anomalies | BotNet Beaconing
 [HIGH  ] 192.168.101.36
           New Country Destinations | New Destination IPs / ASNs
           | External Destination Fan-out
 [HIGH  ] 192.168.101.41        DNS Anomalies | BotNet Beaconing
 [HIGH  ] 192.168.101.72
           New Country Destinations | New Destination IPs / ASNs
           | External Destination Fan-out
 [HIGH  ] 192.168.101.125
           New Country Destinations | New Destination IPs / ASNs
           | External Destination Fan-out
 [HIGH  ] 192.168.101.201       DNS Anomalies | BotNet Beaconing
 [HIGH  ] 192.168.101.207
           HTTPS Data Exfiltration | External Destination Fan-out
           | DNS Anomalies

 -- MEDIUM confidence (single rule) --
 [MEDIUM] 188.83.72.61          Anomalous External Users
 [MEDIUM] 188.83.72.64          Anomalous External Users
 [MEDIUM] 188.83.72.174         Anomalous External Users
 [MEDIUM] 188.83.72.182         Anomalous External Users
 [MEDIUM] 188.83.72.210         Anomalous External Users
 [MEDIUM] 192.168.101.11        New Source IPs
 [MEDIUM] 192.168.101.14        HTTPS Data Exfiltration
 [MEDIUM] 192.168.101.26        HTTPS Data Exfiltration
 [MEDIUM] 192.168.101.68        New Destination IPs / ASNs
 [MEDIUM] 192.168.101.93        New Source IPs
 [MEDIUM] 192.168.101.117       BotNet Beaconing
 [MEDIUM] 192.168.101.138       New Destination IPs / ASNs
 [MEDIUM] 192.168.101.148       DNS Anomalies
 [MEDIUM] 192.168.101.157       BotNet Beaconing
 [MEDIUM] 192.168.101.186       New Destination IPs / ASNs
 [MEDIUM] 192.168.101.187       HTTPS Data Exfiltration
 [MEDIUM] 192.168.101.188       BotNet Beaconing
 [MEDIUM] 192.168.101.197       HTTPS Data Exfiltration
 [MEDIUM] 192.168.101.208       HTTPS Data Exfiltration
```

False Negative Check

A false negative is a compromised device that the pipeline did not flag, the most dangerous failure mode in a detection system. To verify that no real anomaly was silently missed, we independently recomputed the highest metric value observed among all unflagged devices for each volume-based rule, and compared it against the threshold. If any unflagged device sits suspiciously close to the boundary, the threshold may be too conservative.

Rule	Highest unflagged value	Threshold	Gap
HTTPS upload	105.2 MB	116.6 MB	11.4 MB
DNS flows	1307 flows	1399 flows	92 flows
BotNet HTTPS interval std	18.44 s (.160, regularisation=1.17×)	18.81 s	0.37 s above floor
BotNet DNS interval std	63.26 s (.16)	63.73 s	0.47 s above floor
External fan-out	234 unique IPs (.114)	258 unique IPs	24 IPs below threshold
New Source IP	N/A — binary invariant	N/A	No false negatives possible by construction

After analyzing the results, no false negatives were found. Every unflagged device sits outside its threshold with a meaningful gap.

The BotNet rows deserve a specific note. For HTTPS beaconing, .160 sits 0.37 s above the 18.81 s floor, but its regularisation ratio is only 1.17×: it was already similarly tight in training, so the observed regularity is a stable baseline characteristic rather than anomalous behaviour. For DNS beaconing, the closest unflagged device has a DNS interval std of 63.26 s (0.47 s above the floor) and a DNS flow count below the volume threshold, so the timing alone would not constitute meaningful beaconing regardless. For the fan-out rule, .114 at 234 unique external IPs is the closest unflagged device, sitting 24 IPs below the 258 threshold with no other anomalous signals, confirming the boundary is not cutting too close to normal behaviour. The New Source IP rule cannot produce false negatives by design: every IP present in test but absent from training is flagged unconditionally, with no threshold to miscalibrate.

Highest Confidence Detections

The 7 HIGH confidence devices are those flagged by two or more completely independent detection rules. Independence here is not just statistical — each rule operates on a different metric, a different protocol, and a different aspect of behaviour.

- The BotNet rule measures traffic timing regularity per protocol.

- The DNS rule measures query volume and destination.
- The HTTPS rule measures upload volume.
- The Geo rule measures destination countries.
- The New Destination rule measures new ASNs and new internal connections.
- The Fan-out rule measures the count of unique external destinations contacted.
- The New Source IP rule checks set membership against training — no metric, no threshold.

None of these share an input or a computation path. When two or more of them agree on the same device, they are making the same accusation from entirely different angles.

IP	Rules Triggered	Interpretation
192.168.101.41	DNS Volume + BotNet	39,493 DNS queries at an exact 0.05 s interval — extreme C2 beaconing via internal DNS relay, confirmed by both volume and timing
192.168.101.23	DNS Volume + BotNet	8,651 DNS queries at 0.05 s — same C2 pattern, smaller scale, same malware family
192.168.101.201	DNS Volume + BotNet	2,941 DNS queries at exact 0.05 s intervals confirmed independently by volume analysis (8.4σ above mean) and timing regularity — strong single-protocol C2 beaconing
192.168.101.207	HTTPS Exfiltration + Fan-out + DNS Anomalies	119 MB uploaded over HTTPS (3σ above threshold); 273 unique external IPs contacted (3.3σ above mean); 1,418 DNS queries at an exact 0.05 s beacon interval — three independent rules each confirming a different facet of the same compromise
192.168.101.72	New Geo + New Destination + Fan-out	Contacted 15 new countries including RU, IR, UA; 354 new-to-network ASNs pointing at Russian hosting infrastructure; 811 unique external IPs (14.9σ) — geographic, infrastructure, and volume expansion confirmed by three independent rules
192.168.101.36	New Geo + New Destination + Fan-out	904 flows to 17 new countries including RU, IR, IQ, KZ; 1,015 flows to 287 new-to-network ASNs; 639 unique external IPs (11.2σ) — same three-rule fingerprint as .72, consistent with coordinated exfiltration campaign

IP	Rules Triggered	Interpretation
192.168.101.125	New Geo + New Destination + Fan-out	523 flows to 11 new countries including RU, IR, UA; 598 flows to 169 new-to-network ASNs; 409 unique external IPs (6.2σ) — third device in the coordinated group, confirmed by the same three independent rules

Attack Patterns

Grouping the 26 flagged devices by behaviour reveals nine distinct attack patterns active in the test period:

Pattern	Devices	Description
Mass HTTPS Exfiltration	.187, .14, .208	4.4–7.6 GB uploaded in a single day, 183–316σ above baseline — bulk data dumps, likely automated
Moderate HTTPS Exfiltration	.197, .26, .207	119–259 MB uploaded, 3–9σ above the 116.6 MB threshold
DNS C&C Beaconing	.41, .23, .201, .148	Exact 0.05 s query interval to internal resolvers — malware heartbeat routed through internal DNS relay
HTTPS C&C Beaconing	.117, .157, .188	Check-in intervals clustering between 1.00–1.03 s across three independent devices — single malware family, shared hardcoded timer
Exfiltration + C2 Beacon	.207	HTTPS exfiltration (119 MB, 3σ) running alongside a DNS C2 beacon (1,418 queries at 0.05 s intervals) and anomalous fan-out to 273 unique external IPs (3.3σ) — three independent rules each confirming a different facet of the same compromise
Lateral Movement	.68, .138, .186	Internal-to-internal TCP:443 traffic to IPs never seen as destinations in training; symmetric byte ratios ($\sim$1:1) inconsistent with web browsing — peer-to-peer command relay or lateral file transfer between compromised hosts

Pattern	Devices	Description
Geo / Infrastructure Expansion	.36, .72, .125	Traffic to countries and ASNs never seen in training, including Russian hosting providers (REG.RU, Cloud.ru, OBIT), plus anomalous fan-out to 409–811 unique external IPs — three independent rules (geo, ASN, volume) each confirming the same coordinated campaign
Rogue Device / New Implant	.11, .93	Devices with zero training presence generating 1,508 and 3,788 flows in the test period — stealthy profiles (14–38 MB HTTPS, 199–502 DNS queries) calibrated to stay below every volume threshold; caught exclusively by the binary set membership check
External Upload Anomaly	.61, .64, .174, .182, .210	External clients breaking the tight 8.50 ratio invariant — three uploading proportionally more than normal, two downloading more than normal

---

SIEM Integration

Sending the Alert

Once all anomalous IPs are identified, each one is reported to the Wazuh manager by sending a UDP syslog message in the format "Alarm UEBA <ip>" — exactly as prescribed by the spec. The implementation uses a raw Python socket, requiring no external tool or Wazuh agent on the monitoring machine:

```python
def send_syslog_alert(ip: str) -> None:
  with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sock.sendto(f"Alarm UEBA {ip}".encode(), (WAZUH_IP, WAZUH_PORT))
```

One packet is sent per flagged IP. All 26 were successfully delivered to 172.100.0.12:514:

```
[*] Sending UEBA alerts to Wazuh (172.100.0.12:514)...
      → Alarm UEBA 192.168.101.11
      → Alarm UEBA 192.168.101.93
      → Alarm UEBA 188.83.72.174
      ...
      → Alarm UEBA 192.168.101.188   (26 total)
```

Decoder and Rule

For Wazuh to interpret the incoming syslog message, a custom decoder and rule were added inside the wazuh.manager container.

The decoder (/var/ossec/etc/decoders/local_decoder.xml) identifies the message by its prefix and extracts the flagged IP into the srcip field:

```
<decoder name="ueba_alarm">
    <prematch>Alarm UEBA</prematch>
  </decoder>

  <decoder name="ueba_alarm_fields">
    <parent>ueba_alarm</parent>
    <regex offset="after_parent">(\d+\.\d+\.\d+\.\d+)</regex>
    <order>srcip</order>
  </decoder>
```

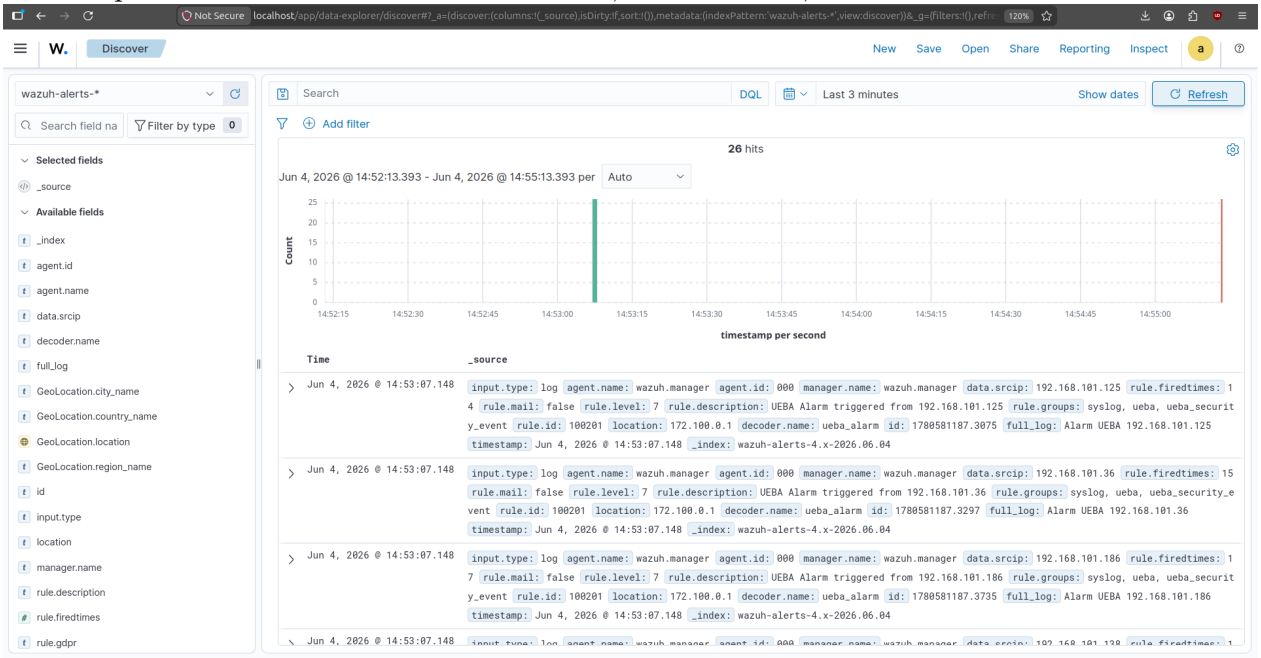
The rule (/var/ossec/etc/rules/local_rules.xml) fires a level 7 alert every time this decoder matches:

```
  <group name="syslog,ueba,">
    <rule id="100201" level="7">
      <decoded_as>ueba_alarm</decoded_as>
      <description>UEBA Alarm triggered from $(srcip)</description>
      <group>ueba_security_event</group>
    </rule>
  </group>
```

In Wazuh's rule classification, level 7 corresponds to "bad word matching" (a keyword or pattern match on log content). It sits above the default log threshold of level 3, ensuring the alert is generated, indexed, and visible in the Security Events dashboard without requiring a severity escalation to a higher level. Rule ID 100201 is the one used having in mind the Guides.

Dashboard

With the Wazuh stack running and all 26 syslog packets delivered, navigating to Security Events and applying the DQL filter rule.id: 100201 shows one event per flagged IP: 26 hits, each carrying data.srcip with the anomalous device's address, rule.level = 7, and decoder.name = ueba_alarm.



Conclusion

This integration pattern, demonstrates that any monitoring system can feed alerts into a SIEM without requiring a Wazuh agent on the monitoring machine. The Wazuh manager acts as a centralised collection point: once the decoder and rule are in place, any source that speaks syslog can contribute structured, queryable events to the same dashboard used for all other security monitoring.